LedgerLine

Secure Financial Transaction Management

Problem Understanding & Development Plan

Correct - Authorized - Auditable

Status: planning stage | Implementation: not started

8	Development phases
7	Financial guarantees
3	User roles
8	Automated test areas

1. Project Overview

LedgerLine is a secure financial transaction management system. It manages bank-style accounts, processes money movements (deposits, withdrawals and transfers) and keeps reliable financial records of everything that happens.

Its primary objective is simple to state and hard to achieve: **every financial operation must be correct, authorized and auditable.**

Correct
Balances always add up. No money appears or vanishes, and no account goes below zero.

Authorized
Only the right person, with the right role, can do each action. The server enforces it.

Auditable
Every movement leaves a permanent record that can be traced and verified later.

2. Problem Understanding

Financial systems fail in surprisingly ordinary ways: a user double-clicks "Pay", a network drops halfway through a transfer, two people withdraw from the same account at the same instant, or someone opens an account that is not theirs. LedgerLine is designed so that each of these situations is handled safely.

Risk	Everyday example	How LedgerLine protects against it
Money created or lost	A transfer debits A but the credit to B never happens.	Double-entry ledger plus a balance-conservation test.
Duplicate requests	A mobile app retries a timed-out payment and money is sent twice.	Idempotency keys: the same key never runs twice.
Overdrafts	Withdrawing 150 from an account holding 100.	Balance checked inside the database transaction.
Concurrency errors	Two simultaneous withdrawals both see the old balance.	Row locking / safe isolation level.
Unauthorized access	User 1 views or drains User 2's account.	Backend authentication, role checks and ownership checks.
Silent tampering	Someone edits or deletes an old ledger row to hide a payment.	Append-only ledger; fixes are compensating transactions.
Inconsistent data after failure	Server crashes between updating a balance and writing the record.	Atomic transactions: all changes commit together or roll back.

The system must ensure (checklist of guarantees):

- [ ] Money cannot be accidentally created or lost.
- [ ] Duplicate requests cannot execute the same financial operation multiple times.
- [ ] Accounts cannot be overdrawn.
- [ ] Concurrent transactions cannot produce incorrect balances.
- [ ] Users cannot access accounts without permission.
- [ ] Financial records cannot be silently modified or deleted.
- [ ] Failed operations must not leave inconsistent financial data.

Why this matters: in finance, a rare bug is still a real loss. These guarantees are built into the design from the first phase, not added at the end.

3. System Architecture (Big Picture)

Three kinds of users reach LedgerLine through a web front end. Every request goes through the backend, which checks identity, permissions and input before any service touches the database. The database enforces consistency using ACID transactions.

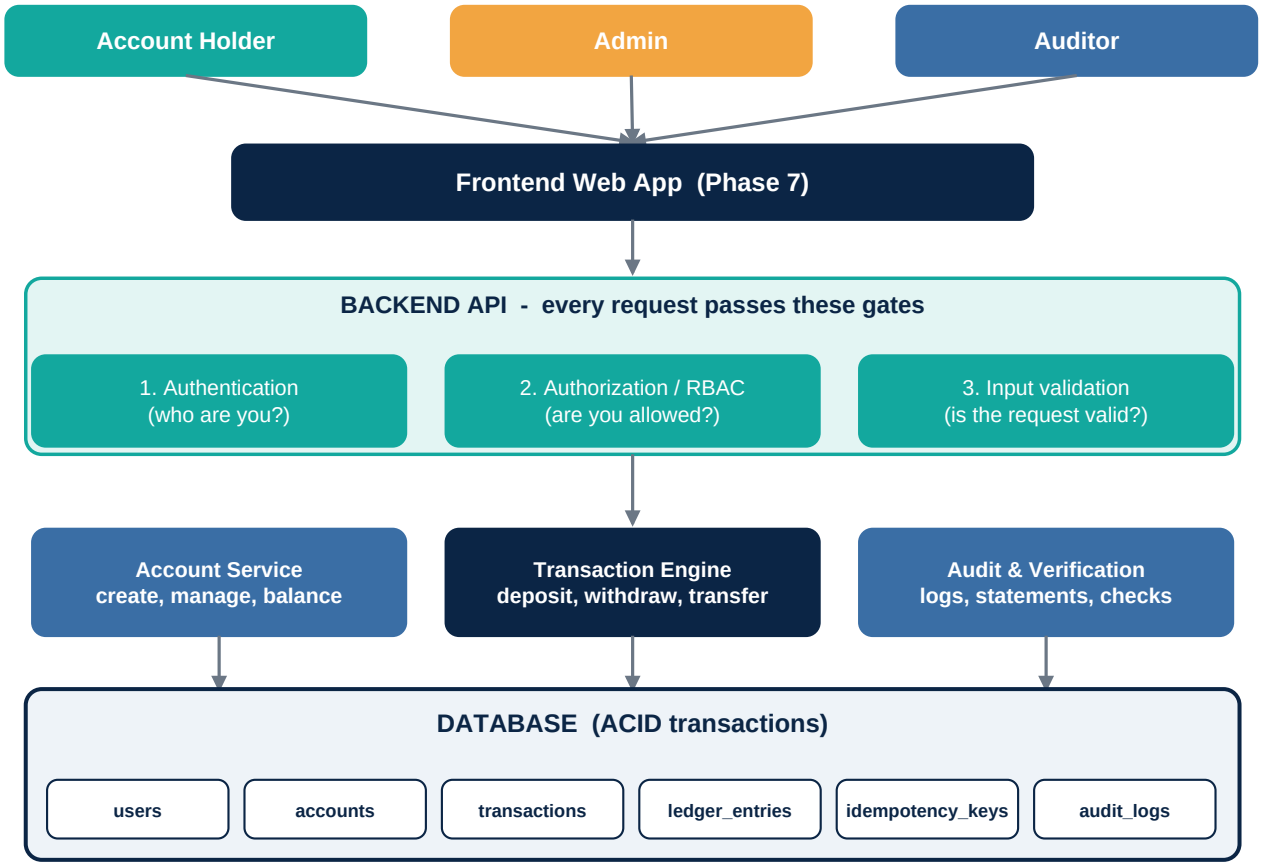

Figure 1 - Proposed layered architecture (conceptual design)

4. Functional Requirements

- [x] Authentication and role-based access
- [x] Account creation and management
- [x] Account balance inquiry
- [x] Deposits and withdrawals
- [x] Transfers between accounts
- [x] Transaction history and statements
- [x] Administrative account management
- [x] Auditor access to financial records
- [x] Audit logging and transaction verification

Ticked boxes mark features in the planned scope of the application.

5. Security Requirements and Roles

Security is enforced **on the backend**, never only in the user interface. The table below shows how the three roles are expected to differ. The planning document names the roles but not every permission, so this matrix is a **proposed** starting point to confirm in Phase 3.

Permission (proposed)	Account Holder	Admin	Auditor
View own accounts and balance	✓	✗	✗
Deposit, withdraw, transfer on own accounts	✓	✗	✗
View own history and statements	✓	✗	✗
Create and manage accounts (administrative)	✗	✓	✗
Read all financial records (read-only)	✗	✗	✓
View audit logs and verify transactions	✗	✓	✓
Move money from accounts they do not own	✗	✗	✗

Security checklist from the plan:

- [ ] Authenticate users before allowing protected operations.
- [ ] Enforce authorization on the backend.
- [ ] Verify account ownership for account-holder operations.
- [ ] Apply appropriate permissions for Admin and Auditor roles.
- [ ] Validate all financial inputs (amounts, account IDs, currency).
- [ ] Protect sensitive credentials and configuration (no secrets in code).
- [ ] Record important security-sensitive operations in the audit log.

Threat documentation: security threats and their mitigations will be recorded in **THREATS.md**.

6. Financial Integrity Requirements

The five integrity requirements are the heart of the project. They work together, as the transfer workflow on the next page shows.

Requirement	Plain-language meaning
Atomic transactions	Balance updates and ledger entries succeed together or fail together. Nothing is ever half-done.
Double-entry ledger	Every transfer records a debit on one account and a matching credit on the other.
Idempotency	Repeating a request with the same idempotency key never runs the operation twice.
Concurrency protection	Simultaneous withdrawals and transfers are serialized safely, so no overdraft or wrong balance can occur.
Immutable records	Ledger rows are never edited or deleted. Mistakes are fixed with new compensating transactions.

Double-entry ledger in action

Example: account A sends 500.00 to account B. Two ledger rows are written in the same database transaction, and the totals must balance.

Entry	Transaction	Account	Debit	Credit
L-101	T-001	A (source)	500.00	
L-102	T-001	B (destination)		500.00
Total			**500.00**	**500.00**

Balance check	Account A	Account B	Sum of both
Before	1,000.00	200.00	1,200.00
After	500.00	700.00	1,200.00

The sum is unchanged: this is the "financial balance conservation" test.

Immutable records and corrections

If a transfer was made by mistake, the original rows stay untouched. A new, linked transaction reverses it, so the full history remains visible.

Entry	Description	Account A	Account B
L-101 / L-102	Original transfer (kept forever)	Debit 500.00	Credit 500.00
L-103 / L-104	Compensating transaction: reversal of T-001	Credit 500.00	Debit 500.00

Transfer workflow - the path of one transfer request

Figure 2 - Flowchart of a transfer from request to commit

Concurrency protection

Two requests arriving at the same instant must not both succeed when only one can. Locking inside a database transaction forces them to take turns.

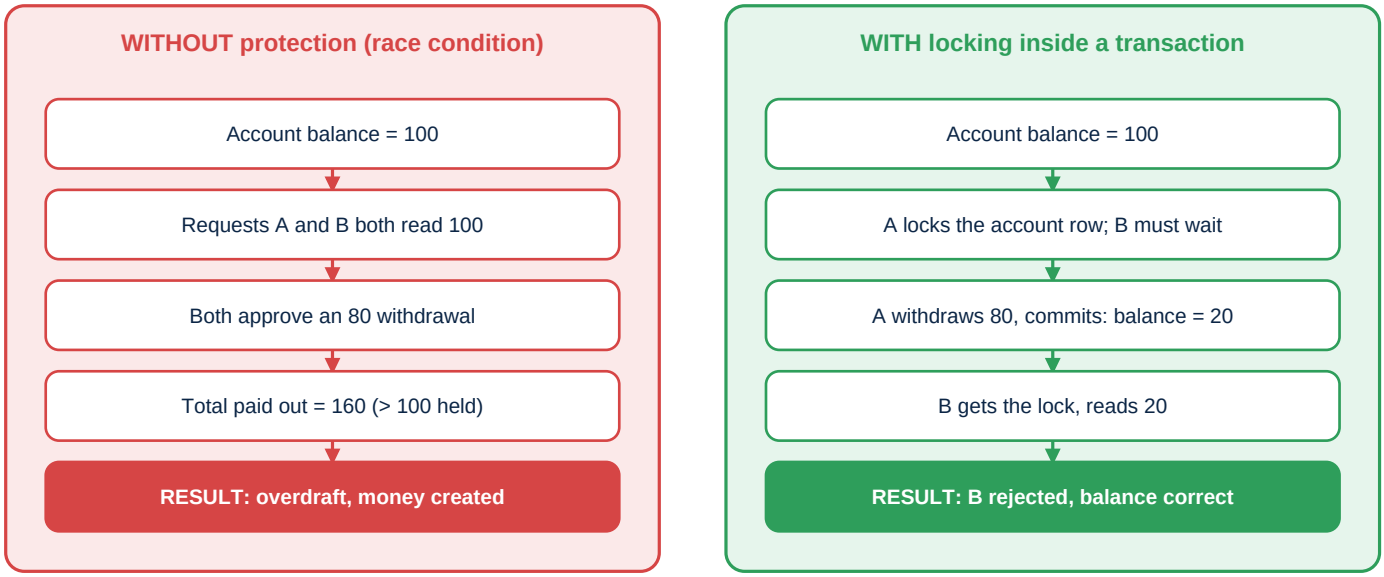

Figure 3 - Race condition vs. protected withdrawal

7. Suggested Data Model

The document describes behavior rather than tables. This diagram shows one reasonable way to store the data, so each requirement has a clear home in the database. Treat it as a proposal for Phase 2.

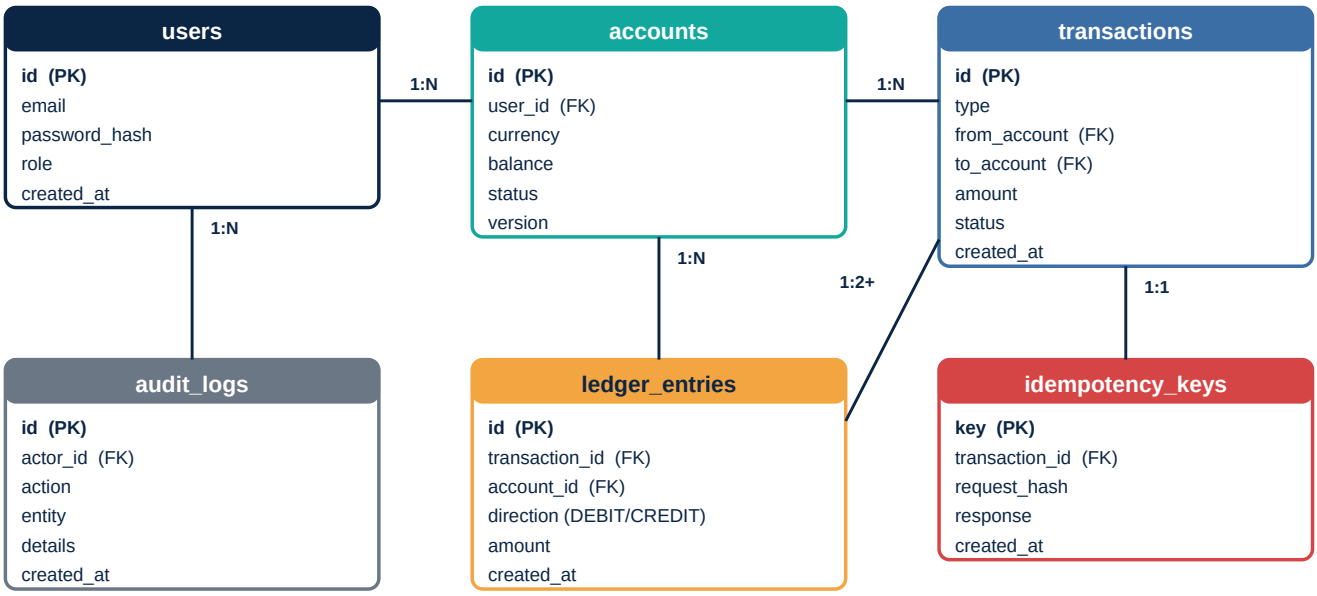

Figure 4 - Proposed entity relationships (not final)

8. Development Phases

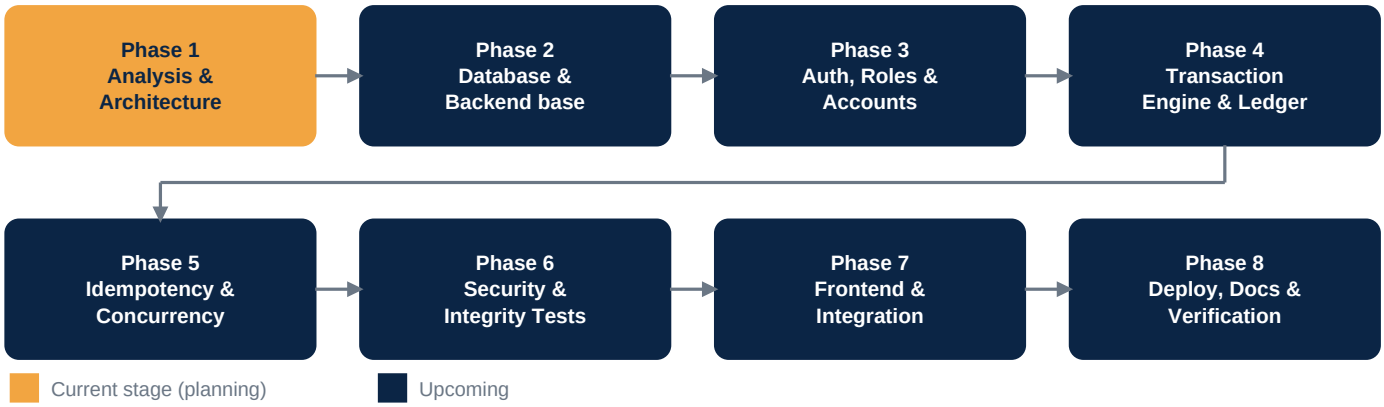

Figure 5 - Eight-phase roadmap

Phase	Focus	Key outcome	Done
1	Problem analysis and architecture	Shared understanding and plan	☐
2	Database schema and backend foundation	Tables, project structure, config	☐
3	Authentication, roles, account management	Login, RBAC, account CRUD	☐
4	Financial transaction engine and ledger	Deposits, withdrawals, transfers, ledger	☐
5	Idempotency and concurrency protection	Safe retries and parallel requests	☐
6	Security and financial integrity testing	Test suite and THREATS.md	☐
7	Frontend development and integration	Working UI connected to the API	☐
8	Deployment, documentation, final verification	Live app and clear docs	☐

9. Engineering Approach

Development is incremental: each feature goes through the same short loop before the next one starts.

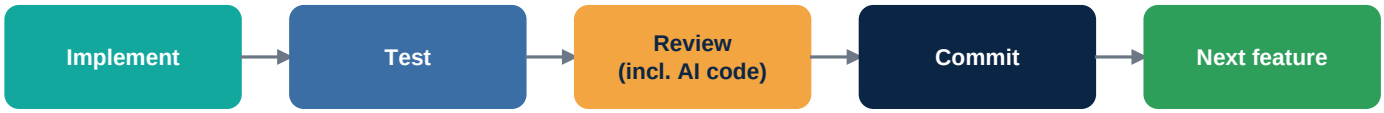

Figure 6 - Per-feature development loop

Practice	What it means
AI-assisted, human-verified	AI coding tools may help write code, but generated code is reviewed and verified before it is accepted.
PROMPTS.md	Records AI mistakes, rejected approaches and corrections.
THREATS.md	Records security threats and the mitigations chosen.
Small, separate commits	Each feature is implemented, tested, reviewed and committed independently.

10. Verification Strategy

Correctness will be proven with automated tests, not assumed. Each area below should have at least one test that fails if the guarantee is broken.

- [ ] **Successful transactions** - valid deposits, withdrawals and transfers update balances correctly.
- [ ] **Insufficient balance rejection** - a withdrawal or transfer larger than the balance is refused.
- [ ] **Duplicate request handling** - the same idempotency key executes only once.
- [ ] **Concurrent withdrawal protection** - parallel withdrawals never overdraw an account.
- [ ] **Unauthorized account access** - users cannot read or move money in accounts they do not own.
- [ ] **Transaction rollback** - a failure mid-operation leaves all data unchanged.
- [ ] **Ledger consistency** - balances always match the sum of ledger entries.
- [ ] **Financial balance conservation** - total money in the system is unchanged by transfers.

11. Definition of Success

- [ ] Financial operations work correctly.
- [ ] Account balances remain consistent with ledger records.
- [ ] Duplicate transactions are prevented.
- [ ] Concurrent operations preserve financial integrity.
- [ ] Authorization is enforced on the server.
- [ ] Automated tests demonstrate correctness.
- [ ] The application is deployed and accessible.
- [ ] Another developer can understand and run the project using its documentation.

12. Current Status

Stage	Problem understanding and development planning
Implementation	Not started

This document establishes the initial understanding and planned engineering approach for LedgerLine.